

03_Introduccion_Spark_executed

March 11, 2026

0.0.1 1.1 ¿Qué es Spark y su Historia?

Apache Spark nació en la Universidad de Berkeley (AMPLab) para solucionar las grandes **limitaciones de Hadoop MapReduce**. MapReduce era excelente, pero muy rígido: escribía datos a disco constantemente, lo que lo hacía terrible para tareas interactivas o algoritmos iterativos (como el Machine Learning).

Spark trajo una revolución al ofrecer un **motor unificado de computación distribuida**. En lugar de tener una herramienta para cada cosa, Spark permite en un solo motor: - **Procesamiento Batch** (por lotes nocturnos, etc.) - **Consultas Interactivas** (mediante SQL y DataFrames) - **Streaming** (Procesamiento en tiempo real con *Structured Streaming*) - **Machine Learning** e **Ingeniería de Grafos**

Todo esto procesando principalmente en **Memoria RAM**, consiguiendo velocidades hasta 100 veces mayores.

0.0.2 1.1 ¿Qué es Spark y su Historia?

Apache Spark nació en la Universidad de Berkeley (AMPLab) para solucionar las grandes **limitaciones de Hadoop MapReduce**. MapReduce era excelente, pero muy rígido: escribía datos a disco constantemente, lo que lo hacía terrible para tareas interactivas o algoritmos iterativos (como el Machine Learning).

Spark trajo una revolución al ofrecer un **motor unificado de computación distribuida**. En lugar de tener una herramienta para cada cosa, Spark permite en un solo motor: - **Procesamiento Batch** (por lotes nocturnos, etc.) - **Consultas Interactivas** (mediante SQL y DataFrames) - **Streaming** (Procesamiento en tiempo real con *Structured Streaming*) - **Machine Learning** e **Ingeniería de Grafos**

Todo esto procesando principalmente en **Memoria RAM**, consiguiendo velocidades hasta 100 veces mayores.

1 Introducción a Apache Spark sobre Hadoop

¡Bienvenido a este cuaderno introductorio! En los cuadernos anteriores aprendimos sobre el ecosistema Hadoop y cómo **HDFS** nos permite almacenar grandes volúmenes de datos de forma distribuida.

Hadoop originalmente utilizaba un motor de procesamiento llamado **MapReduce**. Sin embargo, MapReduce tiene varias limitaciones, principalmente porque lee y escribe constantemente en disco

entre las diferentes fases del mapa (map) y la reducción (reduce). Esto lo hace lento para muchos tipos de tareas analíticas.

Aquí es donde entra **Apache Spark**.

1.1 Hadoop (MapReduce) vs Apache Spark

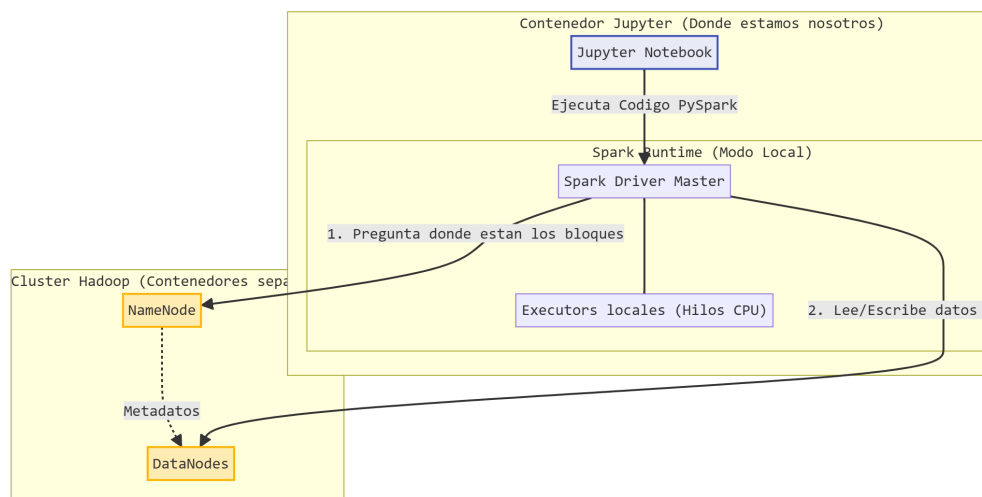
Spark no es un reemplazo de *todo* Hadoop, de hecho, se lleva de maravilla con HDFS y YARN. Spark vino a reemplazar principalmente a la parte de procesamiento: **MapReduce**.

Característica	Hadoop (MapReduce)	Apache Spark
Dónde procesa	En Disco (lee y escribe en HDFS constantemente)	En Memoria RAM (principalmente)
Velocidad	Lento (bueno para procesamiento asíncrono en batch)	Rápido (hasta 100x más rápido en memoria)
Facilidad de uso	Complejo (mucho código Java)	Fácil (Python, Scala, R, SQL, y DataFrames muy parecidos a Pandas)
Casos de Uso	Procesamiento de lotes masivos nocturnos	Análisis interactivo, Machine Learning, Streaming (tiempo real)

1.2 Arquitectura: ¿Dónde entra Spark en nuestro ecosistema?

Spark puede funcionar de varias formas. En empresas gigantescas, Spark se despliega distribuido usando **YARN** para pedirle recursos a los nodos.

Para nuestro entorno de **laboratorio educativo**, hemos optado por la técnica más eficiente en uso de RAM: Ejecutar Spark en **Modo Local** dentro de nuestro propio contenedor de Jupyter, pero **leyendo y escribiendo directamente en el Clúster HDFS de verdad**.



Mira este diagrama para entenderlo:

```

graph TD
    classDef cliente fill:#e8eaf6,stroke:#3f51b5,stroke-width:2px;
    classDef storage fill:#ffe0b3,stroke:#ff9800,stroke-width:2px;

    subgraph Contenedor_Jupyter["Contenedor Jupyter (Donde estamos nosotros)"]
        JN["Jupyter Notebook"]:::cliente
        subgraph Spark_Local["Spark Runtime (Modo Local)"]
            driver["Spark Driver Master"]
            executor["Executors locales (Hilos CPU)"]
            driver --- executor
        end
        JN -->|Ejecuta Código PySpark| driver
    end

    subgraph Cluster_HDFS["Cluster Hadoop (Contenedores separados)"]
        NN["NameNode"]:::storage
        DN["DataNodes"]:::storage
        NN -. ->|Metadatos| DN
    end

    driver -->|1. Pregunta donde estan los bloques| NN
    driver -->|2. Lee/Escribe datos en paralelo| DN

```

De esta forma, **ahorramos muchísima memoria RAM en tu ordenador**, pero aprendes exactamente el mismo código que utilizarías en un clúster de producción con cientos de máquinas.

1.3 Probando Spark

Vamos a inicializar nuestra sesión de Spark. Esta es la puerta de entrada a todas las funciones.

1.3.1 1.4 Componentes Core de Spark

Para entender cómo Spark procesa los datos, es vital conocer sus componentes principales. Aunque en esta sesión usemos un entorno simplificado, la teoría es la misma para clústeres gigantes.

1. Spark Driver (El Director):

- Es el cerebro de la aplicación Spark.
- Analiza, planifica y distribuye el trabajo (tareas).
- Mantiene el estado de la aplicación y presenta la interfaz web (Spark UI).
- *En nuestro laboratorio:* El Driver se ejecuta directamente dentro de este entorno de Jupyter Notebook.

2. Spark Executors (Los Trabajadores):

- Son los procesos que realmente ejecutan el código y procesan los datos.
- Reciben instrucciones del Driver, ejecutan las tareas y devuelven los resultados.
- También almacenan datos en memoria (caché) para que el procesamiento sea rápido.
- *En nuestro laboratorio:* Se ejecutan como hilos (threads) dentro del mismo contenedor de Jupyter.

3. Cluster Manager (El Gestor de Recursos):

- Es el encargado de asignar recursos (CPU, memoria) a las aplicaciones Spark.
- Spark es agnóstico al gestor; puede usar YARN (de Hadoop), Kubernetes, Apache Mesos, o su propio Standalone Cluster Manager.
- *En nuestro laboratorio:* No usamos un Cluster Manager externo. Usamos el **Modo Local** de Spark, que simula todo un clúster en un solo proceso.

1.3.2 1.5 Modo Local vs. Clúster Distribuido

Spark puede ejecutarse en diferentes modos dependiendo de la infraestructura y el objetivo. Es crucial entender la diferencia.

Característica	Modo Local (Nuestro Laboratorio)	Clúster Distribuido (Producción)
Ubicación de Procesos	Driver y Executors corren en una única máquina (o contenedor).	Driver corre en un nodo, Executors corren distribuidos en múltiples nodos .
Gestión de Recursos	Spark mismo gestiona sus hilos. No requiere un sistema externo.	Requiere un Cluster Manager (YARN, K8s) para orquestar contenedores/procesos en distintas máquinas.
Paralelismo	Limitado por los núcleos de la máquina local (ej. <code>local[*]</code> usa todos los cores lógicos disponibles).	Escala horizontalmente: si añades máquinas al clúster, añades más núcleos y RAM para paralelizar.
Fallo de Nodos	Si el proceso falla, la aplicación entera muere.	Tolerancia a fallos real: si un nodo (Executor) cae, el Driver reasigna esa tarea a otro nodo sano.
Uso Principal	Desarrollo, pruebas, aprendizaje, y procesamiento de datos pequeños/medianos que caben en una máquina.	Procesamiento de Big Data real (Terabytes/Petabytes) en entornos de producción.
Ventaja principal	Simplicidad y ligereza. Cero configuración de red, arranca al instante y consume poca RAM base.	Escalabilidad masiva. Puede manejar volúmenes de datos que ninguna máquina sola podría soportar.

Nota importante: El código que escribes en PySpark es **idéntico** en Modo Local y en Clúster Distribuido. Spark abstrae la infraestructura. Puedes desarrollar y probar tu código aquí, y luego llevarlo a un clúster de producción con cientos de máquinas sin cambiar tu lógica.

1.3.3 1.6 APIs de Spark: Bajo vs Alto Nivel

A lo largo de su evolución (desde 2014 hasta hoy), la forma de programar en Spark ha cambiado drásticamente.

1. API Estructurada (Alto Nivel): DataFrames y SQL

- **Es el estándar absoluto hoy en día (Spark 3.x y 4.x).**
 - Un DataFrame es como una tabla estructurada (filas y columnas), muy similar a usar la librería *pandas* o escribir sentencias SQL.
 - **Ventaja:** Spark sabe qué estructura tienen los datos y tiene un optimizador interno mágico llamado *Catalyst* que reescribe tu código para que vuele.
 - **Casos de uso:** El 95% del tiempo. Transformaciones, ETL, lectura/escritura de HDFS/Parquet/CSVs.
2. **API no estructurada (Bajo Nivel): RDDs (Resilient Distributed Datasets)**
- Es la estructura original de Spark. Colecciones de código distribuido sin estructura definida (orientado a objetos).
 - **Ventaja:** Tienes el control absoluto de bajo nivel sobre cada registro y partición.
 - **Desventaja:** Spark no puede optimizarlo tan bien como un DataFrame, y es más engorroso de programar.
 - **Casos de uso:** Rara vez (5%). Solo si trabajas con estructuras sumamente exóticas o lógicas mega-personalizadas que no encajen en una tabla.

1.3.4 1.7 El Cerebro de la Ejecución Distribuida

Para entender por qué tu código vuela o por qué colapsa la memoria, es clave entender este vocabulario:

- **Job (Trabajo):** La gran tarea completa que Spark debe hacer (ej: “Lee este Parquet, agrúpalo y guarda el resultado”).
- **Stage (Etapa):** Spark divide un Job en sub-etapas. ¿Cuándo corta una etapa y empieza otra? Cuando ocurre un **Shuffle** (es decir, cuando los Nodos/Executors necesitan pasarse datos por red entre ellos, como en un `GROUP BY`).
- **Task (Tarea):** La unidad más pequeña de trabajo. Un *Executor* ejecuta una *Task* sobre una *Partición* de datos. ¡A más particiones, más tareas paralelas!
- **DAG (Grafo Acíclico Dirigido):** Es el mapa estricto de pasos que Spark dibuja internamente antes de ejecutar nada para calcular la ruta más óptima.
- **Dependencia Estrecha (Narrow):** Una tarea necesita datos que ya están en el *mismo* nodo (Rápido).
- **Dependencia Ancha (Wide):** Una tarea necesita datos esparcidos por *varios* nodos, obligando a mover datos por la red (Shuffle - Lento).

1.3.5 1.6 APIs de Spark: Bajo vs Alto Nivel

A lo largo de su evolución (desde 2014 hasta hoy), la forma de programar en Spark ha cambiado drásticamente.

1. **API Estructurada (Alto Nivel): DataFrames y SQL**
 - **Es el estándar absoluto hoy en día (Spark 3.x y 4.x).**
 - Un DataFrame es como una tabla estructurada (filas y columnas), muy similar a usar la librería *pandas* o escribir sentencias SQL.
 - **Ventaja:** Spark sabe qué estructura tienen los datos y tiene un optimizador interno mágico llamado *Catalyst* que reescribe tu código para que vuele.
 - **Casos de uso:** El 95% del tiempo. Transformaciones, ETL, lectura/escritura de HDFS/Parquet/CSVs.
2. **API no estructurada (Bajo Nivel): RDDs (Resilient Distributed Datasets)**

- Es la estructura original de Spark. Colecciones de código distribuido sin estructura definida (orientado a objetos).
- **Ventaja:** Tienes el control absoluto de bajo nivel sobre cada registro y partición.
- **Desventaja:** Spark no puede optimizarlo tan bien como un DataFrame, y es más engorroso de programar.
- **Casos de uso:** Rara vez (5%). Solo si trabajas con estructuras sumamente exóticas o lógicas mega-personalizadas que no encajen en una tabla.

1.3.6 1.7 El Cerebro de la Ejecución Distribuida

Para entender por qué tu código vuela o por qué colapsa la memoria, es clave entender este vocabulario:

- **Job (Trabajo):** La gran tarea completa que Spark debe hacer (ej: “Lee este Parquet, agrúpalo y guarda el resultado”).
- **Stage (Etapa):** Spark divide un Job en sub-etapas. ¿Cuándo corta una etapa y empieza otra? Cuando ocurre un **Shuffle** (es decir, cuando los Nodos/Executors necesitan pasarse datos por red entre ellos, como en un gran **GROUP BY**).
- **Task (Tarea):** La unidad más pequeña de trabajo. Un *Executor* ejecuta una *Task* sobre una *Partición* de datos. ¡A más particiones, más tareas paralelas!
- **DAG (Grafo Acíclico Dirigido):** Es el mapa estricto de pasos que Spark dibuja internamente antes de ejecutar nada para calcular la ruta más óptima.
- **Dependencia Estrecha (Narrow):** Una tarea necesita datos que ya están en el *mismo* nodo (Rápido).
- **Dependencia Ancha (Wide):** Una tarea necesita datos esparcidos por *varios* nodos, obligando a mover datos por la red (Shuffle - Lento).

```
[1]: from pyspark.sql import SparkSession

# Inicializamos la sesión de Spark.
# master("local[*]") le dice a Spark que use todos los núcleos disponibles de
# NUESTRO contenedor.
spark = SparkSession.builder \
    .appName("HolaHadoopSpark") \
    .master("local[*]") \
    .getOrCreate()

print(" Spark Initialized!")
print(f"Spark Version: {spark.version}")
```

```
Spark Initialized!
Spark Version: 3.5.0
```

1.3.7 1. Crear datos en memoria distribuidos (DataFrame)

Spark usa **DataFrames**, que son muy similares a los de Pandas, pero preparados para distribuirse en muchos ordenadores bajo el capó (RDBMS).

```
[2]: # Creamos algunos datos en memoria
data = [
    ("Alice", 28, "Ingeniería"),
    ("Bob", 35, "Ventas"),
    ("Carlos", 42, "Marketing"),
    ("Diana", 31, "Ingeniería")
]
columns = ["Nombre", "Edad", "Departamento"]

# Lo convertimos a un Spark DataFrame
df = spark.createDataFrame(data, schema=columns)

# Mostramos el contenido (evaluación perezosa, se ejecuta ahora)
df.show()
```

```
+-----+-----+-----+
|Nombre|Edad|Departamento|
+-----+-----+-----+
| Alice|  28|  Ingeniería|
|  Bob|  35|    Ventas|
|Carlos|  42|  Marketing|
| Diana|  31|  Ingeniería|
+-----+-----+-----+
```

1.3.8 2. Procesamiento (Transformaciones)

Vamos a decirle a Spark que nos diga el promedio de edad por departamento. Fíjate qué parecido es a Pandas o SQL.

```
[3]: import pyspark.sql.functions as F

# Agrupamos por departamento y sacamos la media de edad
df_resumen = df.groupBy("Departamento").agg(F.avg("Edad").alias("Edad_Media"))

df_resumen.show()
```

```
+-----+-----+
|Departamento|Edad_Media|
+-----+-----+
|  Ingeniería|      29.5|
|    Ventas|      35.0|
|  Marketing|      42.0|
+-----+-----+
```

1.3.9 3. Integración con Hadoop (HDFS)

La verdadera magia ocurre al guardar en HDFS. Le diremos a nuestro *Spark Local* que se conecte con el *Namenode* del Clúster Hadoop usando el protocolo `hdfs://namenode:9000`

Nota: Al escribir en HDFS con Spark, a menudo no se crea un solo archivo, sino una carpeta (ej. empleados.parquet) con varios trozos de archivos adentro, lo cual es ideal para lectura paralela.

```
[4]: hdfs_path = "hdfs://namenode:9000/user/alumnos/spark_empleados"

# 1. Guardamos el DataFrame procesado en formato Parquet (el formato rey de Big_
↳Data)
df_resumen.write.mode("overwrite").parquet(hdfs_path)

print(f" Datos guardados con éxito en HDFS en la ruta: {hdfs_path}")
```

Datos guardados con éxito en HDFS en la ruta:
`hdfs://namenode:9000/user/alumnos/spark_empleados`

Podemos verificar desde el propio cuaderno que Spark ha creado esa carpeta en HDFS lanzando un comando de sistema `hdfs`:

```
[5]: !hdfs dfs -ls /user/alumnos/spark_empleados
```

```
Found 2 items
-rw-r--r--    2 root supergroup          0 2026-03-11 10:46
/user/alumnos/spark_empleados/_SUCCESS
-rw-r--r--    2 root supergroup      818 2026-03-11 10:46 /user/alumnos/spark_e
mpleados/part-00000-5e1ba483-b9c9-4c15-b9eb-c14eeaab1a5a-c000.snappy.parquet
```

1.3.10 4. Lectura desde HDFS

Por último, simulamos el proceso de lectura. Esto es lo que harías al inicio de un proyecto de Machine Learning: recuperar datos masivos guardados en Hadoop para procesarlos en Spark.

```
[6]: # Spark lee automáticamente todos los trozos dentro de la carpeta parqueada
df_recuperado = spark.read.parquet(hdfs_path)

print(" Datos recuperados de HDFS:")
df_recuperado.show()
```

Datos recuperados de HDFS:

Departamento	Edad_Media
Ingeniería	29.5
Ventas	35.0
Marketing	42.0


```
[7]: # Siempre es una buena práctica cerrar la sesión de Spark cuando terminas ↵  
      ↪recursos.  
spark.stop()  
print(" Sesión de Spark detenida.")
```

Sesión de Spark detenida.

1.3.11 Recursos Oficiales y Novedades (Spark Moderno)

Si quieres seguir explorando Apache Spark en casa o prepararte para entornos de producción modernos (o instalarlo tú mismo), aquí tienes los recursos oficiales más actualizados:

- **Instalación Rápida Local (Python):** En vez de usar Docker, puedes instalar Spark localmente para pruebas usando pip. `bash pip install "pyspark>=4.1,<4.2"` (*Nota histórica: Aunque originalmente Spark soportaba Scala, Java, Python y R; R está siendo paulatinamente deprecado en las documentaciones recientes de la rama 4.x. Hoy en día, Python (PySpark) y Scala son los reyes absolutos*).
 - **Spark Connect:** Es una nueva arquitectura cliente-servidor introducida recientemente en Spark. Permite que aplicaciones súper ligeras (como tu IDE o un micro-script de Python) se conecten a un clúster remoto de Spark sin tener que empaquetar todo el motor pesado internamente.
 - **Documentación General Oficial:** <https://spark.apache.org/docs/latest/>
 - **Referencia de API para DataFrames (PySpark):** <https://spark.apache.org/docs/latest/api/python/index.html>
-

1.3.12 Recursos Oficiales y Novedades (Spark Moderno)

Si quieres seguir explorando Apache Spark en casa o prepararte para entornos de producción modernos (o instalarlo tú mismo), aquí tienes los recursos oficiales más actualizados:

- **Instalación Rápida Local (Python):** En vez de usar Docker, puedes instalar Spark localmente para pruebas usando pip. `bash pip install "pyspark>=4.1,<4.2"` (*Nota histórica: Aunque originalmente Spark soportaba Scala, Java, Python y R; R está siendo paulatinamente deprecado en las documentaciones recientes de la rama 4.x. Hoy en día, Python (PySpark) y Scala son los reyes absolutos*).
- **Spark Connect:** Es una nueva arquitectura cliente-servidor introducida recientemente en Spark. Permite que aplicaciones súper ligeras (como tu IDE o un micro-script de Python) se conecten a un clúster remoto de Spark sin tener que empaquetar todo el motor pesado internamente.
- **Documentación General Oficial:** <https://spark.apache.org/docs/latest/>
- **Referencia de API para DataFrames (PySpark):** <https://spark.apache.org/docs/latest/api/python/index.html>